

TKO_7092 Evaluation of Machine Learning Methods 2026

Exercise 3

Student name: Shaktiman Choudhury

Student number: 2512876

Student email: shchou@utu.fi

IMPORTANT

This exercise involves using AI. Use only the *Study Chat* service provided by the University of Turku at <https://ai.utu.fi/en>. **Do not use any other AI service!** Before starting, remember to carefully read the guidelines for using AI (<https://intranet.utu.fi/en/sites/ai/guidelines/Pages/default.aspx>) and the terms of use. **Do not share any personal information or copyrighted material with AI.**

Save all your discussions (including your prompts and AI's output) as well as the name of the model you used.

Instructions

The deadline of this exercise is **Wednesday 18 February 2026 at 11:59 PM**. Please contact Juho Heimonen (juaheim@utu.fi) if you have any questions about the exercise. Remember to follow all the general exercise guidelines that are stated in Moodle.

The exercise has several parts, all of which concern the letter below. You will take the role of a data scientist who has been assigned to solve the problem described in the letter. You have an AI tool to assist you, but you alone are responsible for the quality of the solution.

1

Ask AI to write code to solve the task. Analyse which parts of the AI-generated code are correct and which are incorrect. Pay particular attention to the key parts of the cross-validation. You may ask AI to improve the code as many times as you like, as long as you keep analysing its output.

2

Implement the required leave-one-out cross-validations and run your code to get the estimates you were asked to obtain. Here it is okay to use any amount of the AI-generated code you produced above. You can use a complete, fully correct AI-written solution, you can write the implementation from scratch by yourself, or you can take some AI-generated code and complete the implementation manually.

3

Write a report in which you discuss the following:

- Why did the cross-validation described in the letter fail? What is the correct way to do cross-validation here?
- Which parts of the task was AI able to code correctly and which not? Focus particularly on the core of the cross-validation.
- Which parts of the AI-generated code did you use in your implementation? Why? Explain why the selected pieces of code work correctly in your implementation.
- What results did you get with your implementation? Report the estimates and interpret the results in terms of how well the model will work in the situations described in the letter. Explain in detail why your cross-validation is the correct way to estimate the generalisation performance.

Write the report in your own words and explain everything clearly, precisely, and comprehensively. **You are not allowed to use AI to write the report for you** because this is where you show that you have understood the theory and are able to apply it. If you use AI as a teacher (i.e. to explain things to you for learning purposes), you must attach the discussions and clearly state what and how you learnt from the AI. **If there is uncertainty about how the text was produced, you may be required to explain the content of your report in a face-to-face meeting.**

4

Submit the following documents to Moodle:

- The discussions with AI (including your prompts and AI's output), as PDF.
- The implementation of your cross-validation, as PDF and as a Jupyter notebook.
- The report, as PDF. (It is okay to integrate the report to the Jupyter notebook.)

Note that it is not enough to just implement the cross-validation correctly to pass this exercise. You must also explain in plain words what you have done and demonstrate that you understand how cross-validation should be performed on pair-input data. Small errors are acceptable, but you will fail this exercise if there are significant error(s) or omission(s) in the report or in the implementation.

Letter from your client

Dear Data Scientist,

I have a long-term research project regarding a specific set of proteins. I am attempting to discover small organic compounds that can bind strongly to these proteins and thus act as drugs. I have already made laboratory experiments to measure the affinities between some proteins and drug molecules.

My colleague is working on another set of proteins, and the objectives of his project are similar to mine. He has recently discovered thousands of new potential drug molecules. He asked me if I could find the pairs that have the strongest affinities among his proteins and drug molecules. Obviously I do not have the resources to measure all the possible pairs in my laboratory, so I need to prioritise. I decided to do this with the help of machine learning, but I have encountered a problem.

Here is what I have done so far: First I trained a K-nearest neighbours regressor with the parameter value $K=10$ using all the 400 measurements I had already made in the laboratory with my proteins and drug molecules. They comprise of 77 target proteins and 59 drug molecules. To estimate the generalisation performance of the model, I then performed a leave-one-out cross-validation. I used C-index and got a stellar score above 90%. Finally I used the model to predict the affinities of my colleague's proteins and drug molecules. The problem is that when I selected the highest predicted affinities and my colleague tried to verify them in the lab, we found that many of them are much lower in reality. My model clearly does not work despite the high cross-validation score. We also tested the model with my proteins against my colleague's drugs (which is another task I would like to use my model for), but the model did not work there either.

Please explain why my estimation failed and how leave-one-out cross-validation should be performed to get reliable estimates. Also, please implement the leave-one-out cross-validation correctly and report the numbers I need. I want to know whether it would be a waste of my colleague's and my resources if we were to use my model any further.

The data I used to create my model is available in the files `input.data`, `output.data` and `pairs.data` for you to use. The first file contains the features of the pairs, whereas the second contains their affinities. The third file contains the identifiers of the drug and target molecules of which the pairs are composed. The files are paired, i.e. the i^{th} row in each file is about the same pair.

Looking forward to hearing from you soon.

Yours sincerely,
Bio Scientist

Report

I will document all of my thought process, experiments and findings in this report, as this task focuses more on my own understanding of this process than the task itself. After going through the letter, I guess that I have to first replicate the process the bio scientist took first. and find out what he/she did wrong.

i logged into the study chat and am asking the openAI GPT 5.1 model to generate some code to do this task, similar to how the bio scientist did first.

Task 1

Interesting output given by the model. the model asked for more details and gave me a code on what it assumed that I wanted from the prompt. I want to try and run the code given by the model to see how it runs. But after that I will add a few lines to it to see if I can reprompt the gen-ai model with the additional details it asked me to provide. I will just add a small comment like #me to the small lines that I add here. after running the code given by the ai it gets a loocv of 83%. I noticed that the ai code doesnt scale the data. that is probably why the bio scientist got 90%+ on the loocv. I will add that to see if I can replicate that result. Nope, after doing that that loocv did not change much, and it is now at 82.9%. I dont know what model parameters were used but I will give up on replicating that results because this is not how cross validation should be done on pair data on the first place. because in this way, the model is just leaving out one row. but in case of pair data, due to the pair feature dependencies the cross validation will have unrealistically high accuracy.

Another interesting thing to note is the ai model asked me to consider leave one protien out and leave one drug out. I will reprompt with saying that I want to do both. and see what code it outputs.

Interesting output again. Now the ai output and given me 2 loocv implementations, 1 where it is leaving a drug out and another where it is leaving a ptotien out before traing the knn Reg model so that the model can be tested on a drug or protien that it has not seen before, rather than test on just a row.

But I know that when working with paired data there can be 4 out of sample observation types. This code does not consider all of them but only type b and type c.

There a possible combination of 4543 samples of which I have data of 400. so there are 4 types of out of sample observation types.

Type A is an object when (D_i, T_j) does not appear as an in-sample obj but both D_i and T_j appears as another observational combination.

Type B is when, in the combination of (D_i, T_j) , D_i is an out of sample object and T_j is an in-sample object and appears as an observation with some other pair.

Type C is when, in the combination of (D_i, T_j) , T_j is an out of sample object and D_i is an in-sample object and appears as an observation with some other pair

Type D is when in the combination of (D_i, T_j) none appear as an insample object.

I have to select the training data accordingly and evaluate the model on all of these types so that the model doesnt have unrealistic evaluation results.

The reason why the bio scientist is getting such high accuracy from the loocv is maybe because he/she is only considering type A. and so far the Ai generated response has also given me type B and C. I will try again to prompt the ai to give me a better response that considers all 4 types. If it still fails to give me a proper solution to the task, I will just write the logic myself.

After giving it the prompt, the ai-model gave a code that I dont understand some parts of. I need to bring that here and run some test prints to see what it is doing. Nevermind. The ai model assumed wrong and just categorized the in sampled observations and failed the task poorly.

I will start on wokring on my own implementation to this problem. I will use AI to help me do this as fast as possible.

```
In [40]: import numpy as np
import pandas as pd
from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import StandardScaler
```

Analyzing outputs from AI Model Below.

```
In [ ]: # ----- 1. Load data -----
# Adjust loading depending on your exact format (e.g., np.loadtxt, genfro

X = np.loadtxt("input.data")      # shape (400, n_features)
y = np.loadtxt("output.data")     # shape (400,)
pairs = np.loadtxt("pairs.data", dtype=str) # if you need the IDs later

print("Data shapes:", X.shape, y.shape, pairs.shape)#me

X = StandardScaler().fit_transform(X)#me

# Ensure shapes
if y.ndim > 1 and y.shape[1] == 1:
    y = y.ravel()

n_samples = X.shape[0]
assert n_samples == y.shape[0], "X and y must have same number of samples

# ----- 2. Define C-index -----

def c_index(y_true, y_pred):
    """
```

```

Concordance index with 0.5 for ties in predictions.
"""
y_true = np.asarray(y_true).ravel()
y_pred = np.asarray(y_pred).ravel()
n = len(y_true)

concordant = 0
discordant = 0
ties = 0

for i in range(n):
    for j in range(i + 1, n):
        if y_true[i] == y_true[j]:
            # Often excluded, but we'll count as a tie
            continue
        diff_true = y_true[i] - y_true[j]
        diff_pred = y_pred[i] - y_pred[j]
        prod = diff_true * diff_pred

        if prod > 0:
            concordant += 1
        elif prod < 0:
            discordant += 1
        else:
            ties += 1

total = concordant + discordant + ties
if total == 0:
    return np.nan # or 0.5, depending on convention

return (concordant + 0.5 * ties) / total

# ----- 3. LOO cross-validation with KNN (k=10) -----

k = 10
y_pred_all = np.zeros(n_samples)

for i in range(n_samples):
    # Indices for training (all except i)
    train_idx = np.arange(n_samples) != i

    X_train, y_train = X[train_idx], y[train_idx]
    X_test = X[~train_idx].reshape(1, -1)

    # Train KNN regressor
    knn = KNeighborsRegressor(n_neighbors=k, metric='euclidean')
    knn.fit(X_train, y_train)

    # Predict for the left-out sample
    y_pred_all[i] = knn.predict(X_test)[0]

# ----- 4. Compute C-index over all samples -----

cidx = c_index(y, y_pred_all)
print("Leave-one-out C-index (sample-wise):", cidx)

```

Data shapes: (400, 67) (400,) (400, 2)

Leave-one-out C-index (sample-wise): 0.8292731829573935

```

In [ ]: # ----- 1. Load data -----
X = np.loadtxt("input.data")          # shape (400, 67)
y = np.loadtxt("output.data")         # shape (400,)
pairs = np.loadtxt("pairs.data", dtype=str) # shape (400, 2); drug_id,

X = StandardScaler().fit_transform(X)#me

if y.ndim > 1:
    y = y.ravel()

n_samples = X.shape[0]
assert X.shape[0] == y.shape[0] == pairs.shape[0]

# ----- 2. C-index -----
def c_index(y_true, y_pred):
    """
    Concordance index with 0.5 weight for prediction ties.
    """
    y_true = np.asarray(y_true).ravel()
    y_pred = np.asarray(y_pred).ravel()
    n = len(y_true)

    concordant = 0
    discordant = 0
    ties = 0

    for i in range(n):
        for j in range(i + 1, n):
            if y_true[i] == y_true[j]:
                # often excluded; here we skip them (no contribution)
                continue
            diff_true = y_true[i] - y_true[j]
            diff_pred = y_pred[i] - y_pred[j]
            prod = diff_true * diff_pred
            if prod > 0:
                concordant += 1
            elif prod < 0:
                discordant += 1
            else:
                ties += 1

    total = concordant + discordant + ties
    if total == 0:
        return np.nan # or 0.5 by convention
    return (concordant + 0.5 * ties) / total

# ----- 3. Helper: generic leave-one-group-out -----
def leave_one_group_out_knn(X, y, group_ids, k=10):
    """
    group_ids: 1D array of length n_samples with group labels
               (e.g., drug IDs or target IDs)
    Returns: predictions (n_samples,) from grouped L00 CV.
    """
    unique_groups = np.unique(group_ids)
    y_pred = np.zeros_like(y, dtype=float)

    for g in unique_groups:

```

```

test_idx = (group_ids == g)
train_idx = ~test_idx

# Skip groups with no training data (shouldn't happen here)
if np.sum(train_idx) == 0:
    continue

X_train, y_train = X[train_idx], y[train_idx]
X_test = X[test_idx]

model = KNeighborsRegressor(n_neighbors=k, metric='euclidean')
model.fit(X_train, y_train)
y_pred[test_idx] = model.predict(X_test)

return y_pred

# ----- 4. Leave-one-drug-out -----
drug_ids = pairs[:, 0]
y_pred_drug = leave_one_group_out_knn(X, y, drug_ids, k=10)
cidx_drug = c_index(y, y_pred_drug)
print("Leave-one-drug-out C-index:", cidx_drug)

# ----- 5. Leave-one-target-out -----
target_ids = pairs[:, 1]
y_pred_target = leave_one_group_out_knn(X, y, target_ids, k=10)
cidx_target = c_index(y, y_pred_target)
print("Leave-one-target-out C-index:", cidx_target)

```

Leave-one-drug-out C-index: 0.5132957393483709

Leave-one-target-out C-index: 0.8291729323308271

```

In [ ]: # ----- 1. Load data -----
X = np.loadtxt("input.data")          # shape (400, 67)
y = np.loadtxt("output.data")         # shape (400,)
pairs = np.loadtxt("pairs.data", dtype=str) # shape (400, 2)

if y.ndim > 1:
    y = y.ravel()

n_samples = X.shape[0]
assert X.shape[0] == y.shape[0] == pairs.shape[0], "Shape mismatch"

# ----- 2. C-index -----
def c_index(y_true, y_pred):
    """
    Concordance index with 0.5 weight for prediction ties.
    Pairs with equal y_true are ignored.
    """
    y_true = np.asarray(y_true).ravel()
    y_pred = np.asarray(y_pred).ravel()
    n = len(y_true)

    concordant = 0
    discordant = 0
    ties = 0

    for i in range(n):

```



```

    for j in range(i + 1, n):
        if y_true[i] == y_true[j]:
            # ignore pairs with equal true values
            continue
        diff_true = y_true[i] - y_true[j]
        diff_pred = y_pred[i] - y_pred[j]
        prod = diff_true * diff_pred
        if prod > 0:
            concordant += 1
        elif prod < 0:
            discordant += 1
        else:
            ties += 1

    total = concordant + discordant + ties
    if total == 0:
        return np.nan
    return (concordant + 0.5 * ties) / total

# ----- 3. L00 with type classification -----

# Prepare containers to collect true/pred values per type
results = {
    "A": {"y_true": [], "y_pred": []},
    "B": {"y_true": [], "y_pred": []},
    "C": {"y_true": [], "y_pred": []},
    "D": {"y_true": [], "y_pred": []},
}

k = 10

for i in range(n_samples):
    # --- 3.1 Define train/test for this fold ---
    test_idx = i
    train_mask = np.ones(n_samples, dtype=bool)
    train_mask[test_idx] = False

    X_train, y_train = X[train_mask], y[train_mask]
    X_test = X[test_idx].reshape(1, -1)
    y_test = y[test_idx]
    d_test, t_test = pairs[test_idx] # Di, Tj for the test pair

    # --- 3.2 Determine which drug/targets are present in training ---
    drugs_train = set(pairs[train_mask, 0])
    targets_train = set(pairs[train_mask, 1])

    # presence flags
    d_in_train = d_test in drugs_train
    t_in_train = t_test in targets_train

    # --- 3.3 Determine type for this test pair ---
    # Type A: (Di, Tj) pair not in training, but both Di and Tj appear wi
    # Type B: Di unseen, Tj seen
    # Type C: Di seen, Tj unseen
    # Type D: Di unseen and Tj unseen
    #
    # Note: (Di, Tj) is never in training because this is L00 on pairs.
    if d_in_train and t_in_train:
        type_label = "A"

```

```

elif (not d_in_train) and t_in_train:
    type_label = "B"
elif d_in_train and (not t_in_train):
    type_label = "C"
else: # (not d_in_train) and (not t_in_train):
    type_label = "D"

# --- 3.4 Train model and predict for this test pair ---
# Edge case: if training set collapses to very few samples, k may be
# Use min(k, n_train) to avoid errors.
n_train = X_train.shape[0]
k_eff = min(k, n_train)

model = KNeighborsRegressor(n_neighbors=k_eff, metric='euclidean')
model.fit(X_train, y_train)
y_pred = model.predict(X_test)[0]

# --- 3.5 Store result under its type ---
results[type_label]["y_true"].append(y_test)
results[type_label]["y_pred"].append(y_pred)

# ----- 4. Compute C-index per type -----
for t in ["A", "B", "C", "D"]:
    y_true_t = np.array(results[t]["y_true"])
    y_pred_t = np.array(results[t]["y_pred"])

    if len(y_true_t) < 2:
        cidx = np.nan
    else:
        cidx = c_index(y_true_t, y_pred_t)

    print(f"Type {t}: n={len(y_true_t)}, C-index={cidx}")

```

Type A: n=398, C-index=0.8295317899320279

Type B: n=0, C-index=nan

Type C: n=2, C-index=1.0

Type D: n=0, C-index=nan

Task 2

my implementation of the Pair-data LOOCV

```

In [41]: #loading the data.
X = pd.read_csv("input.data", sep=r"\s+", header=None)
y = pd.read_csv("output.data", sep=r"\s+", header=None)
pairs = pd.read_csv("pairs.data", sep=r"\s+", header=None, names=["drug_id

```

I didnt want to work with np arrays like the ai generated codes. But I had to ask ai to help me figure out the seperator for .data files.

```

In [42]: # Scaling the features before training the model.
scaler = StandardScaler()

```

```
X_scaled = pd.DataFrame(scaler.fit_transform(X), index=X.index)
```

```
In [43]: #i will write a cindex function similar to the one in the last exercise.
def c_index(y_true, y_pred):
    # Drop missing values
    mask = ~np.isnan(y_pred)
    y_t = y_true[mask].values
    y_p = y_pred[mask]

    n = len(y_t)
    if n < 2:
        return np.nan

    concordant = 0
    ties = 0
    total_pairs = 0

    for i in range(n):
        for j in range(i + 1, n):

            if y_t[i] == y_t[j]:
                continue

            total_pairs += 1
            diff_prod = (y_t[i] - y_t[j]) * (y_p[i] - y_p[j])

            if diff_prod > 0:
                concordant += 1

            elif diff_prod == 0:
                ties += 1

    if total_pairs == 0:
        return np.nan

    return (concordant + 0.5 * ties) / total_pairs
```

I wrote this to be similar to the the one given in the template of the last exercise. But I dont need coordinate function as this task does not have any use of that. The logic should be the same here but I will also ask AI if this is correct or not. So far noerrors on my side. Ai is also happy with this. Alright I will go to the main task now.

```
In [44]: predictions = pd.DataFrame(index=X.index, columns=["A", "B", "C", "D"], d
print(f'Total samples: {X.index.size}')
```

Total samples: 400

```
In [45]: for i in X.index:
    test_drug = pairs.iloc[i,0]
    test_target_protien = pairs.iloc[i,1]

    mask_A = X.index != i
    mask_B = pairs["drug_id"] != test_drug
    mask_C = pairs["target_id"] != test_target_protien
    mask_D = (pairs["drug_id"] != test_drug) & (pairs["target_id"] != tes
```

```

masks = {'A': mask_A, 'B': mask_B, 'C': mask_C, 'D': mask_D}

for col, mask in masks.items():
    X_train = X_scaled[mask]
    y_train = y[mask]

    X_test = X_scaled.iloc[[i]]

    # training model
    knn = KNeighborsRegressor(n_neighbors=10, weights='distance', metric='euclidean')
    knn.fit(X_train, y_train)

    # Predict
    predictions.loc[i, col] = knn.predict(X_test)[0]

```

For every row in the input.data file, I am finding the D_id and the T_id. I am making 4 types to mask sets for every row. In Type A, I am keeping everything other than row I. This means the model will be trained on both D_id and T_id with a combination of some other pair. This is just a traditional LOOCV which the bio scientist probably did as well. In Type B I am keeping all the rows that do not have the same D_id. Basically forcing to the model to predict on unseen data. similar to the type B given by the gen ai before. In Type C, the process is the same but I am masking the T_ids so no same protien from row i will be left in the training set. In type D, I am masking both D_id and T_id. So the training set will not have both the same drug molecule or Target protien in the training set. SO when evaluating the performance, the model has to predict on totally new data.

Finally I need to send the predictions and true labels to the cindex function and print the results.

```

In [46]: results = {
    "Type A": c_index(y, predictions['A']),
    "Type B": c_index(y, predictions['B']),
    "Type C": c_index(y, predictions['C']),
    "Type D": c_index(y, predictions['D']),
}

df_results = pd.DataFrame(list(results.items()), columns=["Scenario", "C-index"])
df_results["C-index"] = (df_results["C-index"]*100).round(2)
print(df_results)

```

	Scenario	C-index
0	Type A	92.74
1	Type B	51.35
2	Type C	92.75
3	Type D	51.36

This is kind of surprising. I thought my logic and code is alright but I am getting a very high Type C. I went to ask AI model to try to explain if there was anything wrong with my logic. I did not understand What it told me was wrong. I think it was trying to tell me that my evalutaion is not per type. Which I do not think is true. I dont know why it said that. Maybe I lack the proper understanding. I will go back to the letter to see I missed some thing. So If I dont find anything new to add I will

continue to answer the next questions. I do not want to make changes because of this Ai response.

Task 3

1. The cross validation mentioned in the letter failed because the bio scientist was only considering the type A or the simple Leave one out cross validation. It is not a good metrics to measure the models performance on pair data. As I stated before, cross validation should be done while considering the 4 out of sample observation types. The letter also mentioned that the model was trying to predict affinities for 2 kinds of tasks. 1st was using protien target to predict affinity with drug molecule and 2nd was using drug molecule features to predict affinity with protien target. I am presuming from the results of my own implementation, the bio scientist did not implement the Type B and Type C LOOCV required for these tasks.
2. When I prompted Ai to complete the task, It made a lot of mistakes and Failed quite badly. Firstly, It assumed that I wanted to only implement a basic LOOCV. even in that, it tried to give me a lot of unnecessary verification codes that were already addressed in the letter. Like checking shape mismatches, etc. But in the second attempt the AI model asked me to implement either leave one drug out or leave one protien out. But in this case I needed to implement both. In this case the AI model did implement type b and Type c but the way it implemented it was not right. Then I also tried explaining to the ai model to seperately evaluate the types for out of sample observations. Then the Ai model did understand the task but the codes it gave me was not the correct implementation of the task. It tried to categorize the data in the 4 types rather than masking and traing the KNN model to evaluate it. After this I just wanted to try to do my own code. But I asked the Ai model if my logic or implementations were correct or not. However, I did not like most of its resopnses.
3. I was making a mistake in the begining with data reading but I solved that with the regex from the AI model. Other than that I was worried about the results from my implementations. So I asked the Ai model about the logics and results but THe outputs that the AI gave were not correct. So i chose to stick to my answers.
4. After implementing the correct way to do leave one out cross validationb for pair data. i got the Cindex for all 4 types. In Type A, the model can predict drugs and protiens that it has seen before well. It has 92%+ accuracy. But in tybe B where I hid the drug from the model, the Cindex was only 51%. which means, the model cannot accuratly predict affinity using protien target with masked drug molecule. But surprisingly, in the Type C, the model is actually working very well. It is getting a Cindex of 92.75%. I think this is because the protien feature might be more similar to each other. So when I am masking one

protien, the model is still able to predict the affinity with the protien from training set. Finally, the type D has also Cindex of 51%. This is the part which I expected the model to perform its worst.

I think the best finding from this is, If the model is tasked with predicting affinities between drugs that it has seen before, and target protien it has not seen before. But it is not usable the other way around. To answer the Question for the bio scientist, He/she and the colleague did not completely waste their time with this project. Because the model can perform this task well but only partially.